Q.1. What is Selenium?

Ans. Selenium is a robust test automation suite that is used for automating web based applications. It supports multiple browsers, programming languages and platforms.

Q.2. What are different forms of selenium?

Ans. Selenium comes in four forms-

1. **Selenium WebDriver -** Selenium WebDriver is used to automate web applications using browser's native methods.
2. **Selenium IDE** - A Firefox plug-in that works on record and play back principle.
3. **Selenium RC -** Selenium Remote Control(RC) is officially deprecated by selenium and it used to work on javascript to automate the web applications.
4. **Selenium Grid -** Allows selenium tests to run in parallel across multiple machines.

Q.3. What are some advantages of selenium?

1. Selenium is open source and free to use without any licensing cost.
2. It supports multiple languages like Java, ruby, python etc.
3. It supports multi browser testing.
4. It has good amount of resources and helping community over the internet.
5. Using selenium IDE component, non-programmers can also write automation scripts
6. Using selenium grid component, distributed testing can be carried out on remote machines possible.

Q.4. What are some limitations of selenium?

1. We cannot test desktop application using selenium.
2. We cannot test web services using selenium.
3. For creating robust scripts in selenium webdriver, programming langauge knowledge is required.
4. We have to rely on external libraries and tools for performing tasks like - logging(log**4**J), testing framework-(testNG, JUnit), reading from external files(POI for excels) etc.

Q.5. Which all browsers/drivers are supported by Selenium Webdriver?

Ans. Some commonly used browsers supported by selenium are-

1. Google Chrome - ChromeDriver
2. Firefox - FireFoxDriver
3. Internet Explorer - InternetExplorerDriver
4. Safari - SafariDriver
5. HtmlUnit (Headless browser) - HtmlUnitDriver
6. Android - Selendroid/Appium
7. IOS - ios-driver/Appium

Q.6. Can we test APIs or web services using Selenium webdriver?

Ans. No, selenium webdriver uses browser's native method to automate the web applications. Since web services are headless, so we cannot automate web services using selenium webdriver.

Q.7. What are the testing type supported by Selenium WebDriver?

Ans. Selenium webdriver can be used for performing automated functional and regression testing.

Q.8. What are various ways of locating an element in selenium?

Ans. The different locators in selenium are-

1. Id
2. XPath
3. cssSelector
4. className
5. tagName
6. name
7. linkText
8. partialLinkText

Q.9. What is an XPath?

Ans. Xpath or XML path is a query language for selecting nodes from XML documents. XPath is one of the locators supported by selenium webdriver.

Q.10. What is an absolute XPath?

Ans. An absolute XPath is a way of locating an element using an XML expression beginning from root node i.e. html node in case of web pages. The main disadvantage of absolute xpath is that even with slightest change in the UI or any element the whole absolute XPath fails.
Example - html/body/div/div[**2**]/div/div/div/div[**1**]/div/input

Q.11. What is a relative XPath?

Ans. A relative XPath is a way of locating an element using an XML expression beginning from anywhere in the HTML document. There are different ways of creating relative XPaths which are used for creating robust XPaths (unaffected by changes in other UI elements).
Example - //input[@id='username']

Q.12. What is the difference between single slash(/) and double slash(//) in XPath?

Ans. In XPath a single slash is used for creating XPaths with absolute paths beginning from root node, Whereas double slash is used for creating relative XPaths.

Q.13. Which XPath you will prefer to use? Why?

Ans. Normally we prefer to use Relative XPath.

2025 Edition

Q.1. What is Selenium?

Ans. Selenium is a robust test automation suite that is used for automating web based applications. It supports multiple browsers, programming languages and platforms.

Q.2. What are different forms of selenium?

Ans. Selenium comes in four forms-

1. **Selenium WebDriver -** Selenium WebDriver is used to automate web applications using browser's native methods.
2. **Selenium IDE** - A Firefox plug-in that works on record and play back principle.
3. **Selenium RC -** Selenium Remote Control(RC) is officially deprecated by selenium and it used to work on javascript to automate the web applications.
4. **Selenium Grid -** Allows selenium tests to run in parallel across multiple machines.

Q.3. What are some advantages of selenium?

1. Selenium is open source and free to use without any licensing cost.
2. It supports multiple languages like Java, ruby, python etc.
3. It supports multi browser testing.
4. It has good amount of resources and helping community over the internet.
5. Using selenium IDE component, non-programmers can also write automation scripts
6. Using selenium grid component, distributed testing can be carried out on remote machines possible.

Q.4. What are some limitations of selenium?

1. We cannot test desktop application using selenium.
2. We cannot test web services using selenium.
3. For creating robust scripts in selenium webdriver, programming langauge knowledge is required.
4. We have to rely on external libraries and tools for performing tasks like - logging(log**4**J), testing framework-(testNG, JUnit), reading from external files(POI for excels) etc.

Q.5. Which all browsers/drivers are supported by Selenium Webdriver?

Ans. Some commonly used browsers supported by selenium are-

1. Google Chrome - ChromeDriver
2. Firefox - FireFoxDriver
3. Internet Explorer - InternetExplorerDriver
4. Safari - SafariDriver
5. HtmlUnit (Headless browser) - HtmlUnitDriver
6. Android - Selendroid/Appium
7. IOS - ios-driver/Appium

Q.6. Can we test APIs or web services using Selenium webdriver?

Ans. No, selenium webdriver uses browser's native method to automate the web applications. Since web services are headless, so we cannot automate web services using selenium webdriver.

Q.7. What are the testing type supported by Selenium WebDriver?

Ans. Selenium webdriver can be used for performing automated functional and regression testing.

Q.8. What are various ways of locating an element in selenium?

Ans. The different locators in selenium are-

1. Id
2. XPath
3. cssSelector
4. className
5. tagName
6. name
7. linkText
8. partialLinkText

Q.9. What is an XPath?

Ans. Xpath or XML path is a query language for selecting nodes from XML documents. XPath is one of the locators supported by selenium webdriver.

Q.10. What is an absolute XPath?

Ans. An absolute XPath is a way of locating an element using an XML expression beginning from root node i.e. html node in case of web pages. The main disadvantage of absolute xpath is that even with slightest change in the UI or any element the whole absolute XPath fails.
Example - html/body/div/div[**2**]/div/div/div/div[**1**]/div/input

Q.11. What is a relative XPath?

Ans. A relative XPath is a way of locating an element using an XML expression beginning from anywhere in the HTML document. There are different ways of creating relative XPaths which are used for creating robust XPaths (unaffected by changes in other UI elements).
Example - //input[@id='username']

Q.12. What is the difference between single slash(/) and double slash(//) in XPath?

Ans. In XPath a single slash is used for creating XPaths with absolute paths beginning from root node, Whereas double slash is used for creating relative XPaths.

Q.13. Which XPath you will prefer to use? Why?

Ans. Normally we prefer to use Relative XPath.

Ralative Xpath can identify element even though some UI changes happed, but can't identify by Absolute Xpath.

Q.14. What is the difference between Absolute XPath and Relative XPath?

Ans. Absolute Xpath will traverse entire HTML from the root node /html.
Relative Xpath directly jump to node based on attribute specified.

Q.15. How can we inspect the web element attributes in order to use them in different locators?

Ans. Using Chropath or developer tools we can inspect the specific web elements. Chropath is a plugin that provides xpaths and CSS Selectors. From automation perspective, "Right click on page inspect element" is used specifically for inspecting web-elements in order to use their attributes like id, class, name etc. in different locators.

Q.16. How can we locate an element by only partially matching its attributes value in Xpath?

Ans. Using **contains()** method we can locate an element by partially matching its attribute's value. This is particularly helpful in the scenarios where the attributes have dynamic values with certain constant part.

xPath expression = //*[contains(@name,'user')]

The above statement will match the all the values of name attribute containing the word 'user' in them.

Q.17. How can we locate elements using their text in XPath?

Ans. Using the **text ()** method

Q.18. How can we move to nth child element using XPath?

Ans. There are two ways of navigating to the nth element using XPath-
Using square brackets with index position
Example - div[**2**] will find the second div element.
Using position()
Example - div[position()=**3**] will find the third div element.

Q.19. What is the syntax of finding elements by class using CSS Selector?

Ans. By **.className** we can select all the element belonging to a particluar class e.g. '.inputtext ' will select all elements having class ' inputtext '.

Q.20. What is the syntax of finding elements by id using CSS Selector?

Ans. By **#idValue** we can select all the elements belonging to a particular id e.g. '#u_0_n' will select the element having id - u_0_n.

Q.21. How can we select elements by their attribute value using CSS Selector?

Ans. Using **[attribute=value]** we can select all the element belonging to a particular attribute e.g. '[type=radio]' will select the element having attribute type of value 'radio'.

Q.22. What is fundamental difference between XPath and css selector?

Ans. The fundamental difference between XPath and css selector is using XPaths we can traverse up in the document i.e. we can move to parent elements. Whereas using CSS selector we can only move downwards in the document.

Q.23. How can we launch different browsers in selenium webdriver?

Ans. By creating an instance of driver of a particular browser-

Q.24. What is the use of driver.get("URL") and driver.navigate().to("URL") command? Is there any difference between the two?

Ans. Both `driver.get("URL")` and `driver.navigate().to("URL")`commands are used to navigate to a URL passed as parameter. There is no difference between the two commands.

Q.25. How can we type text in a textbox element using selenium?

Ans.
```
WebElement searchTextBox = driver.findElement(By.id("search"));
searchTextBox.sendKeys("searchTerm");
```

Q.26. How can we clear a text written in a textbox?

Ans. Using clear() method we can delete the text written in a textbox.
```
driver.findElement(By.id("elementLocator")).clear();
```

Q.27. How to check a checkBox in selenium?

Ans. The same **click()** method used for clicking buttons or radio buttons can be used for checking checkbox as well.

Q.28. How can we submit a form in selenium?

Ans. Using submit() method we can submit a form in selenium.
```
driver.findElement(By.id("form1")).submit();
```
Also, the click() method can be used for the same purpose.

Q.29. Explain the difference between close and quit command.

Ans. **driver.close()** - Used to close the current browser having focus
driver. quit() - Used to close all the browser instances.

Q.30. How to switch between multiple windows in selenium?

Ans. Selenium has `driver.getWindowHandles();` and `driver.switchTo().window("{windowHandleName}")` commands to work with multiple windows.
The `driver.getWindowHandles();` command returns a list of ids corresponding to each window and on passing a particular window handle to `driver.switchTo().window("{windowHandleName}");` command we can switch control/focus to that particular window

```
for(String windowHandle : driver.getWindowHandles())
{
driver.switchTo().window(handle);
}
```

Q.31. What is the difference between driver.getWindowHandle() and driver.getWindow Handles() in selenium?

Ans. `driver.getWindowHandle()` returns a handle of the current page (a unique identifier) Whereas `driver.getWindowHandles()` returns a set of handles of the all the pages available.

Q.32. How can we move to a particular frame in selenium?

Ans. The `driver.switchTo()` commands can be used for switching to frames.
`driver.switchTo().frame("{frameIndex/frameId/frameName}");`
For locating a frame we can either use the index (starting from 0), its name or Id.

Q.33. Can we move back and forward in browser using selenium?

Ans. Yes, using `driver.navigate().back();` and `driver.navigate().forward();` commands we can move backward and forward in a browser.

Q.34. Is there a way to refresh browser using selenium?

Ans. There a multiple ways to refresh a page in selenium-
Using `driver.navigate().refresh()` command
Using `sendKeys(Keys.F5)` on any textbox on the webpage

Q.35. How can we maximize browser window in selenium?

Ans. We can maximize browser window in selenium using following command-
`driver.manage().window().maximize();`

Q.36. How can we fetch a text written over an element?

Ans. Using **getText()** method we can fetch the text over an element.

```
String text = driver.findElement("elementLocator").getText();
```

Q.37. How can we find the value of different attributes like name, class, value of an element?

Ans. Using **`getAttribute("{attributeName}")`** method we can find the value of different attrbutes of an element

e.g.-
```
String valueAttribute = driver.findElement(By.id("elementLocator"))
                                .getAttribute("value");
```

Q.38. How to delete cookies in selenium?

Ans. Using **deleteAllCookies()** method-
```
driver.manage().deleteAllCookies();
```

Q.39. What is an implicit wait in selenium?

Ans. An implicit wait is a type of wait which waits for a specified time while locating an element before throwing ***NoSuchElementException.*** By default selenium tries to find elements immediately when required without any wait. So, it is good to use implicit wait. This wait is applied to all the elements of the current driver instance.
```
driver.manage().timeouts().implicitlyWait(5, TimeUnit.SECONDS);
```

Q.40. What is an explicit wait in selenium?

Ans. An explicit wait is a type of wait which is applied to a particular web element untill the expected condition specified is met
```
WebDriverWait wait = new WebDriverWait(driver, 10);
WebElement element =
wait.until(ExpectedConditions.elementToBeClickable(By.id("elementId")));
```

Q.41. What are some expected conditions that can be used in Explicit waits?

Ans. Some of the commonly used expected conditions of an element that can be used with expicit waits are-

1. elementToBeClickable(WebElement element or By locator)
2. visibilityOfElementLocated(By locator)
3. attributeContains(WebElement element, String attribute, String value)
4. alertIsPresent()
5. titleContains(String title)
6. titleIs(String title)
7. textToBePresentInElementLocated(By, String)

Q.42. What is fluent wait in selenium?

Ans. A fluent wait is a type of wait in which we can also specify polling interval(intervals after which driver will try to find the element) along with the maximum timeout value.
```
Wait<WebDriver> wait = new FluentWait<WebDriver>(driver)
                            .withTimeout(Duration.ofSeconds(30))
```

```
                    .pollingEvery(Duration.ofSeconds(3))
                    .ignoring(NoSuchElementException.class);
WebElement element = wait.until(new Function<WebDriver, WebElement>()
{

        @Override
        public WebElement apply(WebDriver driver)
        {

                WebElement text = driver.findElement(By.xpath(
                        "//span[text()='Select a delivery location to
                        see product availability and delivery options']"));

                if (text.isDisplayed())
                {

                        return text;

                } else
                {

                        return null;

                }

        }

});
```

Q.43. What are the different keyboard operations that can be performed in selenium?

Ans. The different keyboard operations that can be performed in selenium are-

.sendKeys("sequence of characters") - Used for passing character sequence to an input or textbox element.

.pressKey("non-text keys") - Used for keys like control, function keys etc that are non-text.

.releaseKey("non-text keys") - Used in conjunction with keypress event to simulate releasing a key from keyboard event.

Q.44. What are the different mouse actions that can be performed?

Ans. The different mouse events supported in selenium are

1. click(WebElement element)
2. doubleClick(WebElement element)
3. contextClick(WebElement element)
4. moveToEelement(WebElement element)
5. dragAndDrop(source WebElement, target WebElement)

Q.45. Write the code to double click an element in selenium?

Ans.
```
Actions action = new Actions(driver);
WebElement element=driver.findElement(By.id("elementId"));
action.doubleClick(element).build().perform();
```

Q.46. Write the code to right click an element in selenium?

Ans.
```
Actions action = new Actions(driver);
WebElement element=driver.findElement(By.id("elementId"));
action.contextClick(element). build().perform();
```

Q.47. How to mouse hover an element in selenium?

Ans.
```
Actions action = new Actions(driver);
WebElement element = driver.findElement(By.id("elementId"));
action.moveToElement(element).build().perform();
```

Q.48. How to fetch the current page URL in selenium?

Ans. Using **getCurrentURL()** command we can fetch the current page URL-
driver.getCurrentUrl();

Q.49. How can we fetch title of the page in selenium?

Ans. Using `driver.getTitle();`we can fetch the page title in selenium. This method returns a string containing the title of the webpage.

Q.50. How can we fetch the page source in selenium?

Ans. Using `driver.getPageSource();` we can fetch the page source in selenium. This method returns a string containing the page source.

Q.51. How to verify tooltip text using selenium?

Ans. WebElements have an attribute of type 'title'. By fetching the value of 'title' attribute we can verify the tooltip text in selenium.
```
String toolTipText = element.getAttribute("title");
```

Q.52. How to locate a link using its text in selenium?

Ans. Using linkText() and partialLinkText() we can locate a link.
The difference between the two is linkText matches the complete string passed as parameter to the link texts, Whereas partialLinkText matches the string parameter partially with the link texts.
```
WebElement link1 = driver.findElement("By.linkText(“pavantestingtools\")");
WebElement link2 = driver.findElement("By.partialLinkText(“testingtools\")");
```

Q.53. What are DesiredCapabilities in selenium webdriver?

Ans. Desired capabilities are a set of key-value pairs that are used for storing or configuring browser specific properties like its version, platform etc in the browser instances.

Q.54. How can we find all the links on a web page?

Ans. All the links are of anchor tag 'a'. So by locating elements of tagName 'a' we can find all the links on a webpage.

```
List links = driver.findElements(By.tagName("a"));
```

Q.55. What are some commonly encountered exceptions in selenium?

Ans. Some of the commonly seen exception in selenium are-

1. **NoSuchElementException -** When no element could be located from the locator provided.
2. **ElementNotVisibleException -** When element is present in the dom but is not visible.
3. **NoAlertPresentException -** When we try to switch to an alert but the targetted alert is not present.
4. **NoSuchFrameException -** When we try to switch to a frame but the targetted frame is not present.
5. **NoSuchWindowException -** When we try to switch to a window but the targetted window is not present.
6. **TimeoutException -** When a command execution gets timeout.
7. **InvalidElementStateException -** When the state of an element is not appropriate for the desired action.
8. **NoSuchAttributeException -** When we are trying to fetch an attribute's value but the attribute is not correct
9. **WebDriverException -** When there is some issue with driver instance preventing it from getting launched.

Q.56. How can we capture screenshots in selenium?

Ans. Using getScreenshotAs method of TakesScreenshot interface we can take the screenshots in selenium.

```
File scrFile = ((TakesScreenshot)driver).getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(scrFile, new File("D:\\testScreenShot.jpg"));
```

Q.57. How to handle dropdowns in selenium?

Ans. Using Select class-

```
Select dropdown= new Select(driver.findElement(By.id("countries")));
dropdown.selectByVisibleText("India"); //or using index of the option starting  from 0
dropdown.selectByIndex(1);             //or using its value attribute
dropdown.selectByValue("Ind");
```

Q.58. How to check which option in the dropdown is selected?

Ans. Using ***isSelected()*** method we can check the state of a dropdown's option.

```
Select dropdown = new Select(driver.findElement(By.id("countries")));
dropdown.selectByVisibleText("India"); // returns true or false value
System.out.println(driver.findElement(By.id("India")).isSelected());
```

Q.59. How can we check if an element is getting displayed on a web page?

Ans. Using **isDisplayed** method we can check if an element is getting displayed on a web page.

```
driver.findElement(By locator).isDisplayed();
```

Q.60. How can we check if an element is enabled for interaction on a web page?

Ans. Using **isEnabled** method we can check if an element is enabled or not.

```
driver.findElement(By locator).isEnabled();
```

Q.61. What is the difference between driver.findElement() and driver.findElements() commands?

Ans. findElement() returns a single WebElement (found first) based on the locator passed as parameter. Whereas findElements() returns a list of WebElements, all satisfying the locator value passed.

Syntax of findElement():

```
WebElement textbox = driver.findElement(By.id("textBoxLocator"));
```

Syntax of findElements():

```
List elements = driver.findElements("By.id("value")");
```

Another difference between the two is- if no element is found then findElement() throws NoSuchElementException whereas findElements() returns a list of 0 elements.

Q.62. Explain the difference between implicit wait and explicit wait.?

Ans. **An implicit wait,** while finding an element waits for a specified time before throwing *NoSuchElementException* in case element is not found. The timeout value remains valid throughout the webDriver's instance and for all the elements.

```
driver.manage().timeouts().implicitlyWait(180, TimeUnit.SECONDS);
Whereas, Explicit wait is applied to a specified element only-
WebDriverWait wait = new WebDriverWait(driver, 5);
wait.until(ExpectedConditions.presenceOfElementLocated(ElementLocator));
```

Q.63. How can we handle window UI elements and window POP ups using selenium?

Ans. Selenium is used for automating Web based application only(or browsers only). For handling window GUI elements we can use AutoIT or Sikuli.

Q.64. What is Robot API?

Ans. Robot API is used for handling Keyboard or mouse events. It is generally used to upload files

to the server in selenium automation

```
Robot robot = new Robot(); //Simulate enter key action
robot.keyPress(KeyEvent.VK_ENTER);
```

Q.65. How to do file upload in selenium?

Ans. File upload action can be performed in multiple ways-

Using element.sendKeys("path of file") on the webElement of input tag and type file i.e. the elements should be like…

1. Using Robot API.
2. Using AutoIT.
3. Using Sikuli

Q.66. How to handle HTTPS website in selenium? or How to accept the SSL untrusted connection?

Ans. Using profiles in firefox we can handle accept the SSL untrusted connection certificate. Profiles are basically set of user preferences stored in a file.

Firefox

```
FirefoxProfile profile = new FirefoxProfile();
profile.setAcceptUntrustedCertificates(true);
profile.setAssumeUntrustedCertificateIssuer(false);
WebDriver driver = new FirefoxDriver(profile);
```

IE

```
DesiredCapabilities capabilities = new DesiredCapabilities();
capabilities.setCapability(CapabilityType.ACCEPT_SSL_CERTS, true);
System.setProperty("webdriver.ie.driver","IEDriverServer.exe");
WebDriver driver = new InternetExplorerDriver(capabilities);
```

Chrome

```
DesiredCapabilities handlSSLErr = DesiredCapabilities.chrome ();
handlSSLErr.setCapability (CapabilityType.ACCEPT_SSL_CERTS, true);
WebDriver driver = new ChromeDriver (handlSSLErr);
```

Q.67 How to do drag and drop in selenium?

Ans. Using Action class, drag and drop can be performed in selenium. Sample code-

```
Actions act = new Actions(driver);
act.clickAndHold(sourceElement).moveToElement(targetElement).release().build()
.perform();
OR
act.dragAndDrop(source Element, target Element).build().perform();
```

Q.68. How to execute javascript in selenium?

Ans. JavaScript can be executed in selenium using JavaScriptExecuter. Sample code for **javascript execution-**

```
JavascriptExecutor js = ((JavascriptExecutor) driver);
js.executeScript("{Java script code }");
```

Q.69. How to handle alerts in selenium?

Ans. In order to accept or dismiss an alert box the alert class is used. This requires first switching to the alert box and than using accept() or dismiss() command as the case may be.

```
Alert alert = driver.switchTo().alert(); //To accept the alert
alert.accept();

Alert alert = driver.switchTo().alert(); //To cancel the alert box
alert.dismiss();
```

Q.70. What is HtmlUnitDriver?

Ans. HtmlUnitDriver is the fastest WebDriver. Unlike other drivers (FireFoxDriver, ChromeDriver etc), the HtmlUnitDriver is non-GUI, while running no browser gets launched.

Q.71. How to handle hidden elements in Selenium webDriver?

Ans. Using javaScript executor we can handle hidden elements-

```
(JavascriptExecutor(driver)).executeScript("document.getElementsByClassName(El
                                                    ementLocator).click();");
```

Q.72. What is Page Object Model or POM?

Ans. Page Object Model (POM) is a design pattern in selenium. POM helps to create a framework for maintaining selenium scripts.

In POM for each page of the application a class is created having the web elements belonging to the page and methods handling the events in that page.

The test scripts are maintained in separate files and the methods of the page object files are called from the test scripts file.

Q.73. What are the advantages of POM?

Ans. The advantages are POM are-

1. Using POM we can create an Object Repository, a set of web elements in separate files along with their associated functions. There by keeping code clean.
2. For any change in UI (or web elements) only page object files are required to be updated leaving test files unchanged.
3. It makes code reusable and maintainable.

Q.74. What is Page Factory?

Ans. Page factory is an implementation of Page Object Model in selenium. It provides @FindBy annotation to find web elements and **`PageFactory.initElements()`** method to initialize all web elements defined with `@FindBy` annotation.

```
public class SamplePage
	{
		WebDriver driver;
		@FindBy(id="search")
		WebElement searchTextBox;
		@FindBy(name="searchBtn")
		WebElement searchButton;

	//Constructor public samplePage(WebDriver driver)
	{
		this.driver = driver; //initElements method to initialize all
					elements
		PageFactory.initElements(driver, this);
	}
	//Sample method
		public void search(String searchTerm)
		{
		searchTextBox.sendKeys(searchTerm); searchButton.click();
		}
```

Q.75. What is an Object repository?

Ans. An object repository is centralized location of all the objects or WebElements of the test scripts.
In selenium we can create object repository using Page Object Model and Page Factory design patterns.

Q.76. What is a data driven framework?

Ans. A data driven framework is one in which the test data is put in external files like csv, excel etc separated from test logic written in test script files. The test data drives the test cases, i.e. the test methods run for each set of test data values.

Q.77. What is a keyword driven framework?

Ans. A keyword driven framework is one in which the actions are associated with keywords and kept in external files e.g. an action of launching a browser will be associated with keyword - launchBrowser(), action to write in a textbox with keyword - writeInTextBox(webElement, textToWrite) etc.
The code to perform the action based on a keyword specified in external file is implemented in the framework itself.

Q.78. What is a hybrid framework?

Ans. A hybrid framework is a combination of one or more frameworks. Normally it is associated with combination of data driven and keyword driven frameworks where both the test data and test actions are kept in external files(in the form of table).

Q.79. What is selenium Grid?

Ans. Selenium grid is a tool that helps in distributed running of test scripts across different machines having different browsers, browser version, platforms etc in parallel. In selenium grid there is hub that is a central server managing all the distributed machines known as nodes.

Q.80. What are some advantages of selenium grid?

Ans. The advantages of selenium grid are-
It allows running test cases in parallel thereby saving test execution time.
Multi browser testing is possible using selenium grid by running the test on machines having different browsers.
It is allows multi-platform testing by configuring nodes having different operating systems.

Q.81. What is a hub in selenium grid?

Ans. A hub is server or a central point in selenium grid that controls the test executions on the different machines.

Q.82. What is a node in selenium grid?

Ans. Nodes are the machines which are attached to the selenium grid hub and have selenium instances running the test scripts. Unlike hub there can be multiple nodes in selenium grid.

Q.83. Explain the line of code Webdriver driver = new FirefoxDriver();

Ans. In the line of code `WebDriver driver = new FirefoxDriver();`
'WebDriver' is an interface and we are creating an object of type WebDriver instantiating an object of FirefoxDriver class.

Q.84 . What is the purpose of creating a reference variable- 'driver' of type WebDriver instead of directly creating a FireFoxDriver object or any other driver's reference in the statement Webdriver driver = new FirefoxDriver();?

Ans. By creating a reference variable of type WebDriver we can use the same variable to work with multiple browsers like ChromeDriver, IEDriver etc.

Q.85. What is testNG?

Ans. TestNG(NG for Next Generation) is a testing framework that can be integrated with selenium or any other automation tool to provide multiple capabilities like assertions, reporting, parallel test execution etc.

Q.86. What are some advantages of testNG?

Ans. Following are the advantages of testNG:

1. TestNG provides different assertions that helps in checking the expected and actual results.
2. It provides parallel execution of test methods.
3. We can define dependency of one test method over other in TestNG.
4. We can assign priority to test methods in selenium.
5. It allows grouping of test methods into test groups.
6. It allows data driven testing using @DataProvider annotation.
7. It has inherent support for reporting.
8. It has support for parameterizing test cases using @Parameters annotation.

Q.87. What is the use of testng.xml file?

Ans. testng.xml file is used for configuring the whole test suite.
In testng.xml file we can create test suite, create test groups, mark tests for parallel execution, add listeners and pass parameters to test scripts.
Later this testng.xml file can be used for triggering the test suite.

Q.88. How can we pass parameter to test script using testNG?

Ans. Using @Parameter annotation and 'parameter' tag in testng.xml we can pass parameters to the test script.

Q.88. How can we pass parameter to test script using testNG?

Ans. Using @Parameter annotation and 'parameter' tag in testng.xml we can pass parameters to the test script.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">

<suite name="testsuite">
    <test name="sampletest">
    <parameter name="a" value="welcome" />
        <classes>
            <class name="parameterization.Test1" />
        </classes>
    </test>
</suite>
```

```
package parameterization;
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;

public class Test1 {

    @Parameters("a")
    @Test
    public void m1(String s)
    {
        System.out.println("the value from xml file is:" +s);
    }
}
```

Q.89. How can we create data driven framework using testNG?

Ans. Using @DataProvider we can create a data driven framework in which data is passed to the associated test method and multiple iteration of the test runs for the different test data values passed from the @DataProvider method.

The method annotated with @DataProvider annotation return a **2**D array of object.

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class DataProviderExample {

    WebDriver driver;

    @BeforeClass
    void setup()
    {
        System.setProperty("webdriver.chrome.driver","C://Drivers/chromedriver_win32/chromedriver.exe");
        driver=new ChromeDriver();
    }

    @Test(dataProvider="users")
    void loginTest(String uname,String pwd)
    {
        driver.get("http://newtours.demoaut.com/");
        driver.findElement(By.name("userName")).sendKeys(uname);
        driver.findElement(By.name("password")).sendKeys(pwd);
        driver.findElement(By.name("login")).click();
        Assert.assertEquals(driver.getTitle(), "Find a Flight: Mercury Tours:");
    }

    @DataProvider(name="users")
    String [][] logindata()
    {
        String data[][]={{"mercury","mercury"},{"asasa","mercury1"},{"mercury2","mercury2"} };
        return data;
    }

    @AfterClass
    void closebrowser()
    {
    driver.quit();
    }

}
```

Data Provider method

Q.90. What is the use of TestNG Listeners?

Ans.

1. TestNG provides us different kind of listeners using which we can perform some action in case an event has triggered.
2. Usually testNG listeners are used for configuring reports and logging.
3. One of the most widely used lisetner in testNG is ITestListener interface and TestListenerAdapter Class.
4. It has methods like onTestSuccess, onTestFailure, onTestSkipped etc.
5. We need to implement this interface creating a listener class of our own.

Q.91. What is the use of @Listener annotation in TestNG?

Ans. We need to implement ITestListener interface by creating a listener class of our own. After that using the @Listener annotation, we can use specify that for a particular test class, our customized listener class should be used.

Q.91. What is the use of @Listener annotation in TestNG?

Ans. We need to implement ITestListener interface by creating a listener class of our own. After that using the @Listener annotation, we can use specify that for a particular test class, our customized listener class should be used.


```java
package testnglisteners;

import org.testng.ITestContext;
import org.testng.ITestListener;
import org.testng.ITestResult;

public class Mylisteners implements ITestListener {
    @Override
    public void onTestStart(ITestResult result) {
        System.out.println(" test is started");
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        System.out.println(" test is passed");
    }

    @Override
    public void onTestFailure(ITestResult result) {
        System.out.println("test is failed");
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        System.out.println("test is skipped");
    }

    @Override
    public void onTestFailedButWithinSuccessPercentage(ITestResult result) {
        // TODO Auto-generated method stub

    }

    @Override
    public void onStart(ITestContext context) {
        // TODO Auto-generated method stub

    }

    @Override
    public void onFinish(ITestContext context) {
        // TODO Auto-generated method stub

    }
}
```

```java
package testnglisteners;

import org.testng.Assert;
import org.testng.annotations.Listeners;
import org.testng.annotations.Test;

@Listeners(testnglisteners.Mylisteners.class)
public class LoginTest {

    @Test
    void setup()
    {
        Assert.fail();
    }

    @Test
    void loginByEmail()
    {
        Assert.assertTrue(true);
    }

    @Test(dependsOnMethods={"setup"})
    void loginByFacebook()
    {
        Assert.assertTrue(true);
    }
}
```

Q.92. How can we make one test method dependent on other using TestNG?

Ans. Using dependsOnMethods parameter inside @Test annotation in testNG we can make one test method run only after successful execution of dependent test method.

```
@Test(dependsOnMethods = { "preTests" })
```

Q.93. How can we set priority of test cases in TestNG?

Ans. Using priority parameter in @Test annotation in TestNG we can define priority of test cases. The default priority of test when not specified is integer value **0**.

Example: @Test(priority=1)

Q.94. What are commonly used TestNG annotations?

Ans. The commonly used TestNG annotations are-

@Test
@BeforeMethod
@AfterMethod

@BefoerClass
@AfterClass
@BeforeTest
@AfterTest
@BeforeSuite
@AfterSuite

Q.95. What are some common assertions provided by testNG?

Ans. Some of the common assertions provided by testNG are-

1. **assertEquals(String actual, String expected, String message) -** (and other overloaded data type in parameters)
2. **assertNotEquals(double data1, double data2, String message) -** (and other overloaded data type in parameters)
3. **assertFalse(boolean condition, String message)**
4. **assertTrue(boolean condition, String message)**
5. **assertNotNull(Object object)**
6. **fail(boolean condition, String message)**
7. **true(String message)**

Q.96. How can we run test cases in parallel using TestNG?

Ans. In order to run the tests in parallel just add these two key value pairs in suite-

```
parallel="{methods/tests/classes}"
thread-count="{number of thread you want to run simultaneously}".
```

```java
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;

public class CrossBrowserTest {

    WebDriver driver = null;

    @Parameters("browser")
    @Test
    public void launchBrowser(String br) {
        if (br.equals("FF")) {
            // Firefox Browser
            System.setProperty("webdriver.gecko.driver", "C://Drivers/geckodriver-v0.19.1-win64/geckodriver.exe");
            driver = new FirefoxDriver();
        }

        else if (br.equals("IE")) {
            System.setProperty("webdriver.ie.driver", "C://Drivers/IEDriverServer_Win32_3.7.0/IEDriverServer.exe");
            driver = new InternetExplorerDriver();
        }

        else if (br.equals("CH")) {
            System.setProperty("webdriver.chrome.driver", "C://Drivers/chromedriver_win32/chromedriver.exe");
            driver = new ChromeDriver(); // opens the browser
        }

        driver.get("http://newtours.demoaut.com/");

    }

    @Test
    public void verifyTitle() {
        Assert.assertEquals(driver.getTitle(), "Welcome: Mercury Tours");
    }

    /*
     * @Test public void registration() { //selenium code for registration }
     */

    @AfterClass
    public void closeBrowser() {
        driver.quit();
    }

}
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">

<suite name="paralleltesting" parallel="tests" thread-count="5">

    <test name="FirefoxTest">
    <parameter name="browser" value="FF" />
        <classes>
            <class name="parameterization.CrossBrowserTest" />
        </classes>
    </test>


    <test name="ChromeTest">
    <parameter name="browser" value="CH" />
        <classes>
            <class name="parameterization.CrossBrowserTest" />
        </classes>
    </test>


    <test name="IETest">
    <parameter name="browser" value="IE" />
        <classes>
            <class name="parameterization.CrossBrowserTest" />
        </classes>
    </test>

</suite>
```

Q.97. Name an API used for reading and writing data to excel files.

Ans. Apache POI API and JXL (Java Excel API) can be used for reading, writing and updating excel files.

Q.98. Name an API used for logging in Java.

Ans. Log4j is an open source API widely used for logging in Java.
It supports multiple levels of logging like - ALL, DEBUG, INFO, WARN, ERROR, TRACE and FATAL.

Q.99. What is the use of logging in automation?

Ans. Logging helps in debugging the tests when required and also provides a storage of test's runtime behavior.

Q.100. What is InvocationCount in TestNG?

Ans. This is a TestNG attribute that defines number of times a test method should be invoked or executed before executing any other test method.

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.Test;

public class invocationCount {

    @Test(invocationCount = 5)
    public void getTitle() {
        System.setProperty("webdriver.chrome.driver", "C:/Drivers/chromedriver_win32/chromedriver.exe");
        WebDriver driver = new ChromeDriver();
        driver.get("http://www.pavantestingtools.com/");
        driver.manage().window().maximize();
        System.out.println("Website Title: "+driver.getTitle());
        driver.quit();
    }

    @Test
    public void secondTest() {
        System.out.println("This will be executed at the end");
    }
}
```

Q.101. How can we run a Test method multiple times in a loop (without using any data provider)?

Ans. Using invocationCount parameter and setting its value to an integer value, makes the test method to run n number of times in a loop.

```java
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.Test;

public class invocationCount {

   @Test(invocationCount = 5)
    public void getTitle() {
        System.setProperty("webdriver.chrome.driver", "C:/Drivers/chromedriver_win32/chromedriver.exe");
        WebDriver driver = new ChromeDriver();
        driver.get("http://www.pavantestingtools.com/");
        driver.manage().window().maximize();
        System.out.println("Website Title: "+driver.getTitle());
        driver.quit();
    }

    @Test
    public void secondTest() {
        System.out.println("This will be executed at the end");
    }
}
```

Q.102. What is the default priority of test cases in TestNG?

Ans. The default priority of test when not specified is integer value **0**. So, if we have one test case with priority **1** and one without any priority then the test without any priority value will get executed first (as default value will be **0** and tests with lower priority are executed first).

Q.103. What is the difference between soft assertion and hard assertion in TestNG?

Ans. Soft assertions (SoftAssert) allows us to have multiple assertions within a test method, even when an assertion fails the test method continues with the remaining test execution. The result of all the assertions can be collated at the end using **softAssert.assertAll()** method.

Here, even though the first assertion fails still the test will continue with execution and print the message below the second assertion.

Hard assertions on the other hand are the usual assertions provided by TestNG. In case of hard assertion in case of any failure, the test execution stops, preventing execution of any further steps within the test method.

```java
@Test
public void softAssertionTest(){
    SoftAssert softAssert= new SoftAssert();

    //Assertion failing
    softAssert.fail();
    System.out.println("Failing");

    //Assertion passing
    softAssert.assertEquals(1, 1);
    System.out.println("Passing");

    //Collates test results and marks them pass or fail
    softAssert.assertAll();
}
```

Q.104. How to fail a testNG test if it doesn't get executed within a specified time?

Ans. We can use timeOut attribute of @Test annotation. The value assigned to this timeout attribute will act as an upperbound, if test doesn't get executed within this time frame then it will fail with timeout exception.

```java
@Test(timeOut = 1000)
public void timeOutTest() throws InterruptedException {
    //Sleep for 2sec so that test will fail
    Thread.sleep(2000);
    System.out.println("Will throw Timeout exception!");
}
```

Q.105. How can we skip a test case conditionally?

Ans. Using SkipException, we can conditionally skip a test case. On throwing the skipException, the test method marked as skipped in the test execution report and any statement after throwing the exception will not get executed.

```java
@Test
public void testMethod(){
  if(conditionToCheckForSkippingTest)
    throw new SkipException("Skipping the test");
  //test logic
}
```

Q.106. How can we make sure a test method runs even if the test methods or groups on which it depends fail or get skipped?

Ans. Using "alwaysRun" attribute of @Test annotation, we can make sure the test method will

run even if the test methods or groups on which it depends fail or get skipped.
Here, even though the parentTest failed, the dependentTest will not get skipped instead it will executed because of "alwaysRun=true". In case, we remove the "alwaysRun=true" attribute from @Test then the report will show one failure and one skipped test, without trying to run the dependentTest method.

```
@Test
public void parentTest() {
  Assert.fail("Failed test");
}

@Test(dependsOnMethods={"parentTest"}, alwaysRun=true)
public void dependentTest() {
  System.out.println("Running even if parent test failed");
}
```

Q.107. Why and how will you use an Excel Sheet in your project?

Ans. The reason we use Excel sheets is because it can be used as data source for tests. An excel sheet can also be used to store the data set while performing DataDriven Testing.

Q.108. How can you redirect browsing from a browser through some proxy?

Ans. Selenium provides a PROXY class to redirect browsing from a proxy. Look at the example below:

```
String PROXY = "199.201.125.147:8080";

org.openqa.selenium.Proxy proxy = new.org.openqa.selenium.Proxy();
proxy.setHTTPProxy(Proxy)
 .setFtpProxy(Proxy)
 .setSslProxy(Proxy)
DesiredCapabilities cap = new DesiredCapabilities();
cap.setCapability(CapabilityType.PROXY, proxy);
WebDriver driver = new FirefoxDriver(cap);
```

Q.109. How to scroll down a page using JavaScript in Selenium?

Ans. We can scroll down a page by using window.scrollBy() function.
Example: `((JavascriptExecutor)driver).executeScript("window.scrollBy(0,500)");`

Q.110. How to scroll down to a particular element?

Ans. To scroll down to a particular element on a web page, we can use the function scrollIntoView().

Example:

```
((JavascriptExecutor)driver).executeScript("arguments[0].scrollIntoView();",
                                                                    element);
```

Q.111. How to set the size of browser window using Selenium?

Ans. To maximize the size of browser window, you can use the following piece of code:
`driver.manage().window().maximize();` – To maximize the window
To resize the current window to a particular dimension, you can use the **setSize()** method.

```
System.out.println(driver.manage().window().getSize());
Dimension d = new Dimension(420,600);
driver.manage().window().setSize(d);
```

Q.112. Can we enter text without using sendKeys()?

Ans. Yes. We can enter/ send text without using sendKeys() method. We can do it using JavaScriptExecutor.

```
JavascriptExecutor jse = (JavascriptExecutor) driver;
jse.executeScript("document.getElementById('Login').value=Test text without sendkeys");
```

Q.113. Explain how you will login into any site if it is showing any authentication popup for username and password?

Ans. Since there will be popup for logging in, we need to use the explicit command and verify if the alert is actually present. Only if the alert is present, we need to pass the username and password credentials.
The sample code:

```
WebDriverWait wait = new WebDriverWait(driver, 10);
Alert alert = wait.until(ExpectedConditions.alertIsPresent());
alert.authenticateUsing(new UserAndPassword(**username**, **password**));
```

Q.114. Explain what is Group Test in TestNG?

Ans. In TestNG, methods can be categorized into groups. When a particular group is being executed, all the methods in that group will be executed. We can execute a group by parameterizing it's name in group attribute of @Test annotation.
Example: `@Test(groups={"aaa"})`

```java
public class GroupTestExample {

    @Test(groups = { "sanity" })
    public void loginByemail() {
        System.out.println(" this is login by email");
    }
    @Test(groups = { "sanity" })
    public void loginByfacebook() {
        System.out.println(" this is login by facebook");
    }
    @Test(groups = { "sanity" })
    public void loginBytwitter() {
        System.out.println(" this is login by twitter");
    }
    @Test(groups = { "sanity", "regression" })
    public void signupbyemail() {
        System.out.println("signup by email");
    }
    @Test(groups = { "sanity", "regression" })
    public void signupbyfacebook() {
        System.out.println("signup by facebbok");
    }
    @Test(groups = { "sanity", "regression" })
    public void signupbytwitter() {
        System.out.println("signup by twitter");
    }
    @Test(groups = { "regression" })
    public void paymentReturnbybank() {
        System.out.println("payment return by bank");
    }
    @Test(groups = { "regression" })
    public void payementindollar() {
        System.out.println("this is payement by dollar method");
    }
    @Test(groups = { "regression" })
    public void payementinrupees() {
        System.out.println("this is payement by rupees method");
    }
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd" >

<suite name="Sample Suite">
  <test name="testing">
    <groups>
      <run>
        <include name="regression"/>
        <include name="sanity" />
      </run>
    </groups>

    <classes>
      <class name="GroupingTests.GroupTestExample" />
    </classes>
  </test>
</suite>
```

Q.115. How To Run Failed Test Cases Using TestNG In Selenium WebDriver

Ans. By using "testng-failed.xml"

Q.116. What is Stale Element Exception? How to handle it?

Ans. Stale means old, decayed, no longer fresh.

Stale Element means an old element or no longer available element.

Assume there is an element that is found on a web page referenced as a WebElement in WebDriver. If the DOM changes then the WebElement goes stale. If we try to interact with an element which is staled then the *StaleElementReferenceException* is thrown. When this happens you will need to refresh your reference, or find the element again.

Q.117. What are different XPath functions that you have used in your Project?

Ans. Contains()

Using OR & AND

Start-with() function

Text()

Q.118. What will happen in background when execute new FirefoxDriver() ?

Ans. Firefox binary will be triggered and Fiefox browser will open with default options.

FirefoxDriver object is created.

Q.119. What is the below statement means and Why?

WebDriver driver = new FirefoxDriver();

Ans. WebDriver is an interface which contain several abstract methods such as get(...), findElamentBy(...) etc.
We simply create reference of web Driver and we can assign objects (Firefox driver, ChromeDriver, IEDriver, Andriod driver etc) to it.

Q.120. How do you handle inner Frames and Adjacent Frames?

Ans. SwitchTo frame**1**, SwitchTo frame**2** (inner frame) work on the element and switchto default content
Use SwitchTo frame to move the control inside frame.

Q.121. How to click on an element which is not visible using selenium WebDriver?

Ans. We can use JavascriptExecutor to click.

```
WebElement element = driver.findElement(By.id("gbqfd"));
JavascriptExecutor executor = (JavascriptExecutor)driver;
executor.executeScript("arguments[0].click();", element);
```

Q.122. Difference between verify and assert?

Ans. **Assert:** Assert command checks if the given condition is true or false. If the condition is true, the program control will execute the next phase of testing, and if the condition is false, execution will stop and nothing will be executed.
Verify: Verify command also checks if the given condition is true or false. It doesn't halts program execution i.e. any failure during verification would not stop the execution and all the test phases would be executed.

Q.123. What is the use of @FindBy annotation?

Ans. @FindBy is used to identify element in the Page Factory approach.

Q.124. Do you use Thread.sleep?

Ans. Rarely

Q.125. What are different pop-ups that you have handle in your projects?

Ans.

1. JavaScript Pop
2. Alert alert = driver.switchTo().alert();
3. Browser Pop Ups
4. Browser Profiles, Robot Class, AutoIT, Sikuli
5. Native OS Pop Ups
6. Browser Profiles, Robot Class, AutoIT, Sikuli

Q.126. How do you handle HTTP Proxy Authentication pop ups in browser?

Ans. Form authentications URL - http://UserName:Password@Example.com

Example:

http://the-internet.herokuapp.com/basic_auth

https://admin:admin@the-internet.herokuapp.com/basic_auth

Q.127. How do you handle Ajax dropdowns?

Ans. With help of Selenium Sync commands like ImplicitWait, WebDriverWait or FluentWait.

Q.128. What is the default port for Selenium Grid?

Ans. 4444

Q.129. How to run tests in multiple browser parallel?

Ans. Using selenium grid

Q.130. How to find broken images in a page using Selenium Web driver.

Ans. Get xpath and then using tag name 'a'; get all the links in the page.

Use HttpURLConnector class and sent method GET

Get the response code for each link and verify if it is **404/500**

```
List<WebElement> links = driver.findElements(By.tagName("a"));

            for (int i = 0; i < links.size(); i++) {
                  WebElement element = links.get(i);

            // By using "href" attribute, we could get the url of the requried link
                  String url = element.getAttribute("href");

            //System.out.println(url);
                  URI link = new URI(url);

            // Create a connection using URL object (i.e., link)
                  HttpURLConnection httpConn = (HttpURLConnection)
                  link.openConnection();

            // Set the timeout for 2 seconds
                  httpConn.setConnectTimeout(2000);

            // connect using connect method
                  httpConn.connect();

            // use getResponseCode() to get the response code.
            if (httpConn.getResponseCode() >= 400) {
                System.out.println(url + " - " + "is Broken Link");
                }
            else {
                   System.out.println(url + " - " + "is valid Link");
                }
```

Q.131. How to disable cookies in browser?

Ans. Using deleteAllVisibleCookies() in selenium.

Q.132. How does u handle dynamic elements without using XPath?

Ans. By using classname or css.

Q.133. Write down scenarios which we can't automate?

Ans. Barcode Reader, Captcha etc.

Q.134. How do you manage the code versions in your project?

Ans. Using SVN, GitHub or other versioning tools.

Q.135. How to count total no of hyperlinks in a page?

Ans.
```
List alllinks=driver.findElements(By.tagName("a"));
System.out.println(alllinks.size());
```

Q.136. What are the benefits of Automation Testing?

Ans.

1. Saves time and money. Automation testing is faster in execution.
2. Reusability of code.
3. Create one time and execute multiple times with less or no maintenance.
4. Easy reporting.
5. It generates automatic reports after test execution.
6. Easy for compatibility testing.
7. It enables parallel execution in the combination of different OS and browser environments.
8. Low-cost maintenance.
9. It is cheaper compared to manual testing in a long run.
10. It is mostly used for regression testing.
11. Supports execution of repeated test cases.
12. Minimal manual intervention.
13. Test scripts can be run unattended.
14. Maximum coverage.
15. It helps to increase the test coverage.

Q.137. What type of tests have you automated?

Ans. Our main focus is to automate test cases to do Regression testing, Smoke testing, and Sanity testing. Sometimes based on the project and the test time estimation, we do focus on End to End testing.

Q.138. How many test cases you have automated per day?

Ans. It depends on Test case scenario complexity and length.
I did automate **2-5** test scenarios per day when the complexity is limited.
Sometimes just **1** or fewer test scenarios in a day when the complexity is high.

Q.139. What is Selenium IDE?

Ans.

1. Selenium IDE (Integrated Development Environment) is a Firefox plugin.
2. It is the simplest framework in the Selenium Suite.
3. It allows us to record and playback the scripts. Even though we can create scripts using
4. Selenium IDE, we need to use Selenium WebDriver to write more advanced and robust test cases.

Q.140. What is Selenese?

Ans. Selenese is the language which is used to write test scripts in Selenium IDE.

Q.141. What is Selenium RC?

Ans. Selenium RC (Selenium **1**).
Selenium RC was the main Selenium project for a long time before the WebDriver merge brought up Selenium **2**.
Selenium **1** is still actively supported (in maintenance mode). It relies on JavaScript for automation. It supports Java, Javascript, Ruby, PHP, Python, Perl and C#. It supports almost every browser out there.

Q.142. What is Selenium WebDriver?

Ans.

1. Selenium WebDriver (Selenium **2**) is a browser automation framework that accepts commands and sends them to a browser.
2. It is implemented through a browser-specific driver.
3. It controls the browser by directly communicating with it.
4. Selenium WebDriver supports Java, C#, PHP, Python, Perl, Ruby.

Q.143. When do you use Selenium Grid?

Ans. Selenium Grid can be used to execute same or different test scripts on multiple platforms and browsers concurrently so as to achieve distributed test execution.

Q.144. What are the advantages of Selenium Grid?

Ans.

1. It allows running test cases in parallel thereby saving test execution time.
2. It allows multi-browser testing

3. It allows us to execute test cases on multi-platform

Q.145. What is a hub in Selenium Grid?

Ans. A hub is a server or a central point that controls the test executions on different machines.

Q.146. What is a node in Selenium Grid?

Ans. Node is the machine which is attached to the hub. There can be multiple nodes in Selenium Grid.

Q.147. What are the types of WebDriver APIs available in Selenium?

Ans.

1. Firefox Driver
2. InternetExplorer Driver
3. Chrome Driver
4. HTMLUNIT Driver
5. Opera Driver
6. Safari Driver
7. Android Driver
8. iPhone Driver

Q.148. Which WebDriver implementation claims to be the fastest?

Ans. The fastest implementation of WebDriver is the HTMLUnitDriver. It is because the HTMLUnitDriver does not execute tests in the browser.

Q.149. What are the Programming Languages supported by Selenium WebDiver?

Ans.

1. Java
2. C#
3. Python
4. Ruby
5. Perl
6. PHP

Q.150. What are the Operating Systems supported by Selenium WebDriver?

Ans.

1. Windows
2. Linux
3. Mac

Q.151. What are the Open-source Frameworks supported by Selenium WebDriver?

Ans.

1. JUnit
2. TestNG
3. CUCUMBER
4. JBHEAVE

Q.152. What is the super interface of WebDriver?

Ans. SearchContext.

Q.153. What are the types of waits available in Selenium WebDriver?

Ans. In Selenium we could see three types of waits such as Implicit Waits, Explicit Waits and Fluent Waits.

Implicit Waits
Explicit Waits
Fluent Waits
PageLoadTimeOut
Thread.sleep() – static wait

Q.154. How to clear the text in the text box using Selenium WebDriver?

Ans. By using clear() method

```
WebDriver driver = new FirefoxDriver();
driver.get("https://www.gmail.com");
driver.findElement(By.xpath("xpath_of_element1")).sendKeys("Software Testing
                                                                    ");
driver.findElement(By.xpath("xpath_of_element1")).clear();
```

Q.155. How to get a text of a web element?

Ans. By using getText() method

Q.156. How to get an attribute value using Selenium WebDriver?

Ans. By using getAttribute(value);

Q.157. List some scenarios which we cannot automate using Selenium WebDriver?

Ans.

1. Bitmap comparison Is not possible using Selenium WebDriver
2. Automating Captcha is not possible using Selenium WebDriver
3. We can not read bar code using Selenium WebDriver
4. windows OS based pop ups
5. third party calendars/element
6. Image and Word/PDF

Q.158. How can you use the Recovery Scenario in Selenium WebDriver?

Ans. By using "Try Catch Block" within Selenium WebDriver Java tests.

```
try
{
        driver.get("www.xyz.com");
}
catch (Exception e)
{
        System.out.println(e.getMessage());
}
```

Q.159. Database testing in Selenium?

Ans. We can use JDBC driver to connect to any database in Java.

Q.160. How to schedule the Test Suite Execution?

Ans. We can schedule the test suite execution using CI tools like hudson(Jenkins), Bamboo. Alternatively, we can use windows scheduler to launch the test execution.

Q.161. How to send an email stating the execution status to all stakeholders in Selenium?

Ans. We can send mail in Java using javax.mail library.

Q.162. What is desired capabilities?

Ans. Capabilities are used to set the values of the browser attributes before we launch any browser using selenium web driver.

Q.163. Version control tools like SVN, GIT?

Ans. We use version control tools like gitHub/SVN to track the changes to the files in a project and work in collaboration.

Q.164. Build tools - Ant, Maven?

Ans. We use these tools to manage build activities for the Java project.

Q.165. CI tools - Jenkin, Bamboo?

Ans. These are continuous integration tools helping in quick deployment of applications, testing t hem and reporting the issues in the code before it is too late. It helps in getting the application into production quickly and with more quality confidence.

```
public class AddTwoMatrix
{
	public static void main(String args[])
	{
		int m, n, c, d;

		int first[][] = { { 1, 2 }, { 5, 10 }, { 2, 6 } };
		int second[][] = { { 2, 6 }, { 1, 2 }, { 5, 3 } };

		m = first.length;
		n = first[0].length;

		int sum[][] = new int[m][n];

		System.out.println("Calculating Sum of 2 matrices....");

		for (c = 0; c < m; c++)
			for (d = 0; d < n; d++)
				sum[c][d] = first[c][d] + second[c][d]; // replace '+'
									with '-' to subtract matrices

		System.out.println("Sum of 2 matrices....");

		for (c = 0; c < m; c++)
		{
			for (d = 0; d < n; d++)
				System.out.print(sum[c][d] + "\t");

			System.out.println();
		}
	}

}
```

```
public class ArrayListExample1
{
	public static void main(String[] args)
	{

		// Declaration
		ArrayList list = new ArrayList();

		// Add values to arraylist
		list.add("John");
		list.add("David");
		list.add("Scott");
		list.add("Smith");

		System.out.println(list.size()); // returns size of arraylist
```

```
		// reading values from arraylist
		for (String s : list)
		{
			System.out.println(s);
		}

	}

}
```

```
public class EvenOrOddNumber
{
	public static void main(String[] args)
	{

		int num = 10;

		if (num % 2 == 0)
		{
			System.out.println("Number is even number");
		}

		else
		{
			System.out.println("Number is odd number");
		}
	}

}
```

```
public class EvenOrOddNumber
{
	public static void main(String[] args)
	{

		int num = 10;

		if (num % 2 == 0)
		{
			System.out.println("Number is even number");
		}

		else
		{
			System.out.println("Number is odd number");
		}
	}

}
```

```
public class BinarySearch
{
	public static void main(String args[])
	{
		int c, first, last, middle, n, search_element;

		int array[] = { 100, 200, 300, 400, 500 };

		search_element = 200;

		n = array.length;

		first = 0;
		last = n - 1;
		middle = (first + last) / 2;

		while (first <= last)
		{
			if (array[middle] < search_element)
				first = middle + 1;
			else if (array[middle] == search_element)
			{
				System.out.println(search_element + " found at location "
+ (middle + 1) + ".");
				break;
			} else
				last = middle - 1;

			middle = (first + last) / 2;
		}
		if (first > last)
			System.out.println(search_element + " isn't present in the
                                                        list.\n");
	}

}
```

```
package LogicalPrograms;

public class EvenAndOddNumbersinArray
{
	public static void main(String[] args)
	{

		int a[] = { 10, 20, 15, 3, 6, 7, 8, 2, 5, 7 };

		int n = a.length;

		System.out.print("Odd numbers:");
		for (int i = 0; i < n; i++)
		{
			if (a[i] % 2 != 0)
			{
```

```
                        System.out.print(a[i] + " ");
                }
        }
        System.out.println();

        System.out.print("Even numbers:");
        for (int i = 0; i < n; i++)
        {
                if (a[i] % 2 == 0)
                {
                        System.out.print(a[i] + " ");
                }
        }

    }

}
```

```
import java.util.ArrayList;
public class ArrayListExample2
{
    public static void main(String[] args)
    {

        // Declaration
        ArrayList list = new ArrayList();

        // Adding values to array list
        list.add("welcome");
        list.add(100);
        list.add(10.5);
        list.add('C');
        list.add(true);

        System.out.println(list.size()); // size of arraylist

        System.out.println(list.get(2)); // returns specific value from array
                                         list, index starts from 0

        System.out.println("Before inserting:" + list); // print all the values
                                                           from arraylist

        // Insert values into araylist
        list.add(1, "selenium");
        System.out.println("After insertion:" + list);

        // remove values from arraylist
        list.remove(3);
        System.out.println("After remove:" + list);

        // reading values from array list usign for loop

        for (Object i : list)
```

```
		{
			System.out.println(i);
		}

	}

}
```

```
import java.util.Random;

public class GenerateRandomeNumbersInGivenRange
{
	public static void main(String[] args)
	{
		// Generating random integers between 0 and 50 using Random class

		System.out.println("Random integers between 0 and 50 using Random class
:");

		Random random = new Random();

		for (int i = 0; i < 5; i++)
		{
			System.out.println(random.nextInt(50));
		}

		// Generating random integers between 0 and 50 range using Math.random()

		System.out.println("Random integers between 0 and 50 using Math.random()
:");

		for (int i = 0; i < 5; i++)
		{
			System.out.println((int) (Math.random() * 50));
		}

	}
```

```
import java.util.Arrays;

public class BinarySearchUsingMethod
{

	public static void main(String args[])
	{
		int array[] = { 10, 20, 30, 40, 50 }; // Should be in order

		System.out.println(Arrays.binarySearch(array, 30));
	}
}
```

```java
public class BubbleSort
{
	public static void main(String[] args)
	{
		int n, c, d, temp;

		int array[] = { 500, 300, 200, 400, 100 };
		n = array.length;

		System.out.println("Array Before Bubble Sort");

		for (int i = 0; i < array.length; i++)
		{
			System.out.print(array[i] + " ");
		}

		// Sorting
		temp = 0;

		for (int i = 0; i < n; i++)
		{
			for (int j = 1; j < (n - i); j++)
			{
				if (array[j - 1] > array[j])
				{
					temp = array[j - 1];
					array[j - 1] = array[j];
					array[j] = temp;
				}
			}
		}

		System.out.println();

		System.out.println("Array After Bubble Sort");

		for (int i = 0; i < array.length; i++)
		{
			System.out.print(array[i] + " ");
		}
	}
}
```

```java
import java.util.Scanner;

public class CountTheWords
{
	public static void main(String[] args)
	{
		{
			System.out.println("Enter the string:");

			Scanner sc = new Scanner(System.in);
```

```
                String s = sc.nextLine();

                int count = 1;

                for (int i = 0; i < s.length() - 1; i++)
                {
                        if ((s.charAt(i) == ' ') && (s.charAt(i + 1) != ' '))
                        {
                                count++;
                        }
                }

                System.out.println("Number of words in a string = " + count);
        }

    }

}
```

```
public class CountCharacterOccurence
{
    public static void main(String[] args)
    {
        String s = "Java is java again java again";

        char c = 'a';

        int count = s.length() - s.replace("a", "").length();

        System.out.println("Number of occurances of 'a' is: " + count);
    }
}
```

```
public class GreatestOfThreeNumbers
{
    public static void main(String[] args)
    {

        int a = 50;
        int b = 100;
        int c = 20;

        if (a > b && a > c)
        {
            System.out.println(" a is greatest");
        }
        else if (b > a && b > c)
        {
            System.out.println("b is largest");
        }
        else
```

```
		{
			System.out.println("c is greatest");
		}

	}

}
```

```
public class IfElseCondition
{
	public static void main(String[] args)
	{

		int age = 20;

		if (age >= 18)
		{
			System.out.println("Eligible for vote");
		} else
		{
			System.out.println("NOT Eligible for vote");
		}
	}

}
```

```
{
	public static void main(String[] args)
	{
		// Convert Integer To String Using Integer.toString() Method
		int i = 123;

		String s = Integer.toString(i);

		System.out.println(s);

		// Convert Integer To String Using String.valueOf() method
		s = String.valueOf(i);

		System.out.println(s);

	}

}
```

```
public class LinearSearch
{
	public static void main(String args[])
	{
```

```
		int array[] = { 100, 200, 300, 400, 500 };

		int search_element = 400;

		int c;

		for (c = 0; c < array.length; c++)
		{
			if (array[c] == search_element) // Searching element is present
			{
				System.out.println(search_element + " is present at
							location " + (c + 1) + ".");
				break;
			}
		}
		if (c == array.length) /* Element to search isn't present */
			System.out.println(search_element + " isn't present in array.");
	}

}
```

```
{
	public static void main(String[] args)
	{

		int a = 50;

		int b = 20;

		if (a > b)
		{
			System.out.println("a is largest");
		}

		else
		{
			System.out.println("b is largest");
		}
	}

}
```

```
public class MaxAndMinElementInArray
{
	public static void main(String[] args)
	{

		int array[] = { 10, 100, 20, 50, 5, 60 };

		// Max value in array
		int max = array[0];
```

```
		for (int i = 1; i < array.length; i++)
		{
			if (array[i] > max)
			{
				max = array[i];
			}
		}

		System.out.println("Max Element in array:" + max);

		// Min value in array
		int min = array[0];

		for (int i = 1; i < array.length; i++)
		{
			if (array[i] < min)
			{
				min = array[i];
			}
		}

		System.out.println("Min Element in array:" + min);

	}

}
```

```
public class MultiplicationTable
{
	public static void main(String[] args)
	{

		int day = 5;

		if (day == 1)
		{
			System.out.println("Sunday");
		} else if (day == 2)
		{
			System.out.println("Monday");
		} else if (day == 3)
		{
			System.out.println("Tuesday");
		} else if (day == 4)
		{
			System.out.println("Wednesday");
		} else if (day == 5)
		{
			System.out.println("Thursday");
		} else if (day == 6)
		{
			System.out.println("Friday");
		} else if (day == 7)
```

```
		{
			System.out.println("Saturday");
		} else
		{
			System.out.println("Invalid week number");
		}

	}

}
```

```
public class NumberOfDigits
{
	public static void main(String[] args)
	{

		int count = 0;
		int num = 3452;

		while (num != 0)
		{
			num /= 10; // 345 34 3
			++count;
		}

		System.out.println("Number of digits: " + count);

	}

}
```

```
public class Palindrome
{
	public static void main(String[] args)
	{

		int lastDigit, sum = 0, a;
		int inputNumber = 171; // It is the number to be checked for palindrome

		a = inputNumber;

		// Code to reverse a number
		while (a > 0)
		{
			System.out.println("Input Number " + a);
			lastDigit = a % 10; // getting remainder
			System.out.println("Last Digit " + lastDigit);
			System.out.println("Digit " + lastDigit + " was added to sum " +
(sum * 10));
			sum = (sum * 10) + lastDigit;
			a = a / 10;
```

```
		}

		// if given number equal to sum than number is palindrome otherwise not
		// palindrome
		if (sum == inputNumber)
			System.out.println("Number is palindrome ");
		else
			System.out.println("Number is not palindrome");
	}

}
```

```
public class PalindromeString
{
	public static void main(String[] args)
	{

		String s = "DAD";

		// 1. using for loop
		int len = s.length(); // 7
		String rev = "";

		for (int i = len - 1; i >= 0; i--)
		{
			rev = rev + s.charAt(i); // muineleS
		}

		System.out.println(rev);

		if (s.equals(rev))
		{
			System.out.println("Palindrome string");
		} else
		{
			System.out.println("Not Palindrome string");
		}

		// 2. using StringBuffer class:
		// StringBuffer sf = new StringBuffer(s);
		// System.out.println(sf.reverse());
	}

}
```

```
public class PositiveOrNagitiveNumber
{
	public static void main(String[] args)
	{
```

```
		int num = 10; // positive
		// int num=-10; //Negitive

		if (num > 0)
		{
			System.out.println(" Number is Positive");
		} else
		{
			System.out.println("Number is Negitive");
		}

	}

}
```

```
mport java.util.ArrayList;
import java.util.HashSet;

public class RemoveDuplicatesFromArrayList
{
	public static void main(String[] args)
	{
		// Constructing An ArrayList

		ArrayList listWithDuplicateElements = new ArrayList();

		listWithDuplicateElements.add("JAVA");

		listWithDuplicateElements.add("J2EE");

		listWithDuplicateElements.add("JSP");

		listWithDuplicateElements.add("SERVLETS");

		listWithDuplicateElements.add("JAVA");

		listWithDuplicateElements.add("STRUTS");

		listWithDuplicateElements.add("JSP");

		// Printing listWithDuplicateElements

		System.out.print("ArrayList With Duplicate Elements :");

		System.out.println(listWithDuplicateElements);

		// Constructing HashSet using listWithDuplicateElements

		HashSet set = new HashSet(listWithDuplicateElements);

		// Constructing listWithoutDuplicateElements using set

		ArrayList listWithoutDuplicateElements = new ArrayList(set);
```

```
		// Printing listWithoutDuplicateElements

		System.out.print("ArrayList After Removing Duplicate Elements :");

		System.out.println(listWithoutDuplicateElements);
	}

}
```

```
public class RemoveJunk
{
	public static void main(String[] args)
	{

		String s = "å°?ç±³ä¼“éªŒç‰ˆ latin string 01234567890";
		String s1 = "@#$@#$@ testing #@$@#$@#$ Selenium !@#$@#$@# &&&& Java";

		// Regular Expression: [^a-zA-Z0-9]

		s = s.replaceAll("[^a-zA-Z0-9]", "");
		System.out.println(s);

		s1 = s1.replaceAll("[^a-zA-Z0-9]", "");
		System.out.println(s1);

	}

}
```

```
public class RemoveWhiteSpacesInaString
{
	public static void main(String[] args)
	{
		{
			String str = "  Core Java selenium automation       oops
                                                        programming  ";

			String strWithoutSpace = str.replaceAll("\\s", "");

			System.out.println(strWithoutSpace);
		}

	}

}
```

```
import java.util.Scanner;

public class ReverseChars
{
	public static void main(String[] args)
	{
		Scanner scan = new Scanner(System.in);
		System.out.println("Please enter a string: ");
		String original = scan.nextLine();

		while (original.isEmpty() || original == null)
		{
			System.out.println("Please enter a valid string, empty and null
					strings are not accepted:");
			original = scan.nextLine();
		}
		scan.close();

		ReverseChars output = new ReverseChars();
		String reverseCharacters = output.reverseCharacters(original);
		System.out.println(reverseCharacters);
	}

	private String reverseCharacters(String originalString)
	{
		String reverse = "";

		for (int i = originalString.length() - 1; i >= 0; i--)
		{
			reverse = reverse + originalString.charAt(i);
		}
		return reverse;
	}

}
```

```
public class ReverseEachWord
{
	public static void main(String[] args)
	{
		reverseEachWordOfString("Java Concept Of The Day");
		reverseEachWordOfString("Java J2EE JSP Servlets Hibernate Struts");
		reverseEachWordOfString("I am string not reversed");
		reverseEachWordOfString("Reverse Me");
	}

	static void reverseEachWordOfString(String inputString)
	{
		String[] words = inputString.split(" ");

		String reverseString = "";

		for (int i = 0; i < words.length; i++)
```

```
		{
			String word = words[i];

			String reverseWord = "";

			for (int j = word.length() - 1; j >= 0; j--)
			{
				reverseWord = reverseWord + word.charAt(j);
			}

			reverseString = reverseString + reverseWord + " ";
		}

		System.out.println(inputString);
		System.out.println(reverseString);
		System.out.println("-------------------------");
	}

}
```

```
public class ReverseNumber
{
	public static void main(String args[])
	{
		// 1. using algorithm
		long num = 12345; // 54321
		long rev = 0;

		while (num != 0)
		{
			rev = rev * 10 + num % 10; // 5432
			num = num / 10; // 12
		}

		System.out.println("Reverse num is:" + rev);

		// 2. using StringBuffer method
		long num1 = 12345;
		System.out.println(new StringBuffer(String.valueOf(num1)).reverse());
	}

}
```

```
import java.util.Scanner;

public class ReverseString
{
	public static void main(String[] args) {
		  //Reverse a String:
		   //diff bw String and StringBuffer
		   //do we have reverse function in String?
```

```
        System.out.println("Enter the string:");

        Scanner sc = new Scanner(System.in);

        String s=sc.nextLine();


         //1. using for loop
         int len = s.length(); //8
         String rev = "";

         for(int i =len-1; i>=0; i--)
         {
          rev = rev + s.charAt(i); //muineleS
         }

         System.out.println(rev);

         //2. using StringBuffer class:
         StringBuffer sf = new StringBuffer(s);
         System.out.println(sf.reverse());
    }

}
```

```
public class SearchNumberinArray
{
	public static void main(String[] args)
	{

		int a[] = { 10, 20, 30, 40, 50 };

		int num = 30;
		boolean flag = false;

		for (int i : a)
		{
			if (num == i)
			{
				System.out.println("Element found");
				flag = true;
				break;
			}
		}

		if (flag == false)
		{
			System.out.println("Element NOT found");
		}

	}
```

```
}
```

```
public class SearchNumericValueInArray
{
	public static void main(String[] args)
	{

		int a[] = { 10, 20, 30, 40, 50 };

		int num = 30;
		boolean flag = false;

		for (int i : a)
		{
			if (num == i)
			{
				System.out.println("Element found");
				flag = true;
				break;
			}
		}

		if (flag == false)
		{
			System.out.println("Element NOT found");
		}

	}

}
```

```
public class SearchStringinArray
{
	public static void main(String[] args)
	{

		String a[] = { "abc", "xyz", "pqr", " mno" };

		String search_String = "xyz";

		boolean flag = false;

		for (String s : a)
		{
			if (search_String == s)
			{
				System.out.println("Element found");
				flag = true;
				break;
			}
```

```
		}

		if (flag == false)
		{
			System.out.println("Element NOT found");
		}

	}

}
```

```
package LogicalPrograms;

public class SearchStringValueInArray
{
	public static void main(String[] args)
	{

		String a[] = { "abc", "xyz", "pqr", " mno" };

		String search_String = "xyz";

		boolean flag = false;

		for (String s : a)
		{
			if (search_String == s)
			{
				System.out.println("Element found");
				flag = true;
				break;
			}
		}

		if (flag == false)
		{
			System.out.println("Element NOT found");
		}

	}

}
```

```
import java.util.Arrays;

public class SearchUsingMethod
{
	public static void main(String args[])
	{
		int array[] = { 10, 20, 30, 40, 50 }; // Should be in order

		System.out.println(Arrays.binarySearch(array, 10));
```

```
	}
}
```

```
public class SingleDimArray
{
	public static void main(String[] args)
	{

		int a[] = { 100, 200, 300, 400, 500 }; // Declare an array without size
		                                       and store values

		System.out.println(a.length); // Prints length of an array

		for (int i : a)
		{

			System.out.println(i);
		}

		// How to break for loop in the middle
		for (int i : a)
		{
			if (i == 400)
			{
				break;
			}

			System.out.println(i);
		}

	}

}
```

```
import java.util.Arrays;

public class SortArray
{
	public static void main(String[] args)
	{

		// Number Array sorting
		int data[] = { 4, 10, 2, 6, 1 };

		Arrays.sort(data);

		for (int c : data)
		{
			System.out.println(c);
		}
```

```
		// String array sorting

		String data2[] = { "z", "a", "x" };
		Arrays.sort(data2);

		for (String c : data2)
		{
			System.out.println(c);
		}

	}

}
```

```
import java.util.Arrays;

public class SortingUsingSortMethod
{
	public static void main(String args[])
	{
		int data[] = { 4, 10, 2, 6, 1 };

		Arrays.sort(data);

		for (int c : data)
		{
			System.out.println(c);
		}
	}

}
```

```
public class StringMethods
{
	public static void main(String[] args)
	{

		String s = "welcome";

		// length()
		System.out.println(s.length());

		// concat()
		String s1 = "welcome";
		String s2 = " to java";

		System.out.println(s1.concat(s2));
		System.out.println("welcome".concat(" to java"));

		// trim()
		s = "    welcome    ";
```

```
		System.out.println(s);
		System.out.println(s.trim());

		// charAt()
		s = "Welcome";

		System.out.println(s.charAt(4)); // o

		// contains() --> return true/false
		s = "Welcome to java";
		System.out.println(s.contains("java")); // true
		System.out.println(s.contains("Java")); // false

		// equals() & equalsIgnoreCase()
		s = "Selenium";
		System.out.println(s.equals("SELENIUM"));
		System.out.println(s.equalsIgnoreCase("SELENIUM"));

		// Replace()
		s = "welcome to java";
		System.out.println(s.replace('e', 'a')); // replacing single character
		System.out.println(s.replace("java", "selenium")); // replacing multiple
                                                                   chars

		// substring()
		s = "Welcome";
		System.out.println(s.substring(2, 4)); // lc
		System.out.println(s.substring(4, 7)); // ome

		// toLowerCase() && toUpperCase()

		s = "WelCome";

		System.out.println(s.toLowerCase()); // welcome
		System.out.println(s.toUpperCase()); // WELCOME
	}

}
```

```
public class StringSwapping
{
	public static void main(String[] args)
	{

		// WAP to swap strings without using temp/third variable:

		String a = "Hello";
		String b = "World";

		System.out.println("before swapping: ");
		System.out.println("the value of a is:" + a);
		System.out.println("the value of b is:" + b);
```

```
		// 1. append a and b:
		a = a + b; // HelloWorld

		// 2. Store initial string a in String b:
		b = a.substring(0, a.length() - b.length());

		// 3. Store initial string b in String a:
		a = a.substring(b.length());

		System.out.println("the value of a and b after swapping");

		System.out.println("the value of a is:" + a);
		System.out.println("the value of b is:" + b);

	}

}
```

```
public class SumOfArray
{
	public static void main(String args[])
	{
		int[] array = { 10, 20, 30, 40, 50, 10 };
		int sum = 0;

		// Advanced for loop
		for (int num : array)
		{
			sum = sum + num;
		}
		System.out.println("Sum of array elements is:" + sum);
	}

}
```

```
package LogicalPrograms;

public class SwappingWithoutThirdVariable
{
	public static void main(String[] args)
	{

		int x = 5;
		int y = 10;

		// x = 10, y = 5

		// 1. with using third var : t
		int t;
		t = x; // 5
		x = y; // 10
		y = t; // 5
```

```
		// 2. without using third var: using + operator
		// x = x + y; //15
		// y = x - y; //5
		// x = x - y; // 10

		// 3. without using third var: using * operator
		// x = x * y; //50
		// y = x / y; //5
		// x = x / y; //10

		System.out.println(x);
		System.out.println(y);
	}

}
```

```
public class SwitchCaseStatement
{
	public static void main(String[] args)
	{

		int day = 10;

		switch (day)
		{
		case 1:
			System.out.println("Sunday");
			break;
		case 2:
			System.out.println("Monday");
			break;
		case 3:
			System.out.println("Tuesday");
			break;
		case 4:
			System.out.println("Wednesday");
			break;
		case 5:
			System.out.println("Thursday");
			break;
		case 6:
			System.out.println("Friday");
			break;
		case 7:
			System.out.println("Saturday");
			break;
		default:
			System.out.println("Invalid week number");
		}

	}
```

```
}
```

```
public class TwoDimArray
{
	public static void main(String[] args)
	{

		int a[][] = new int[3][2];

		a[0][0] = 100;
		a[0][1] = 200;

		a[1][0] = 300;
		a[1][1] = 400;

		a[2][0] = 500;
		a[2][1] = 600;

		// int a[][]={ {100,200},{300,400},{500,600}};

		System.out.println(a.length); // return number of rows

		System.out.println(a[0].length); // returns number of columns


		for (int r[] : a)
		{
			for (int c : r)
			{
				System.out.println(c);
			}
		}

	}

}
```

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class WordCountInFile
{

	public static void main(String[] args)
	{
		BufferedReader reader = null;

		// Initializing charCount, wordCount and lineCount to 0
```

```
int charCount = 0;

int wordCount = 0;

int lineCount = 0;

try
{
	// Creating BufferedReader object

	reader = new BufferedReader(new FileReader
				("C:\\SeleniumPractice\\Test.txt"));

	// Reading the first line into currentLine

	String currentLine = reader.readLine();

	while (currentLine != null)
	{
		// Updating the lineCount

		lineCount++;

		// Getting number of words in currentLine

		String[] words = currentLine.split(" ");

		// Updating the wordCount

		wordCount = wordCount + words.length;

		// Iterating each word

		for (String word : words)
		{
			// Updating the charCount

			charCount = charCount + word.length();
		}

		// Reading next line into currentLine

		currentLine = reader.readLine();
	}

	// Printing charCount, wordCount and lineCount

	System.out.println("Number Of Chars In A File : " + charCount);

	System.out.println("Number Of Words In A File : " + wordCount);

	System.out.println("Number Of Lines In A File : " + lineCount);
}
catch (IOException e)
```